

learn-numpy

August 19, 2023

0.1 Numpy (Numerical Python)

- NumPy is the fundamental package for scientific computing in Python. Or you can say its the universal standard for working with numerical data in Python.
- The NumPy library contains multidimensional array and matrix data structures.
- It provides `ndarray`, a homogeneous n-dimensional array object, with methods to efficiently operate on it.
- It can be used to perform a wide variety of mathematical operations on arrays.
- It adds powerful data structures to Python that guarantee efficient calculations with arrays and matrices.

0.2 Difference between a Python list and a NumPy array?

- NumPy arrays are faster and more compact than Python lists.
- NumPy arrays have strict requirements on the homogeneity of the objects.
- For example, a NumPy array of strings can only contain strings and no other data types, but a Python list can contain a mixture of strings, numbers, booleans and other objects.
- NumPy array consumes less memory and is convenient to use i.e. it uses much less memory to store data since it provides a mechanism of specifying the data types. This allows the code to be optimized even further.

There are 6 general mechanisms for creating arrays:

1. Conversion from other Python structures (i.e. lists and tuples)
2. Intrinsic NumPy array creation functions (e.g. `arange`, `ones`, `zeros`, etc.)
3. Replicating, joining, or mutating existing arrays
4. Reading arrays from disk, either from standard or custom formats
5. Creating arrays from raw bytes through the use of strings or buffers
6. Use of special library functions (e.g., `random`)

0.3 Few of the Ways to create NumPy Arrays

1. `np.array()`
2. `np.zeros()`
3. `np.ones()`
4. `np.empty()`
5. `np.arange()`
6. `np.linspace()`

7. np.identity() or np.eye()

```
[1]: # import the module
import numpy as np
```

0.3.1 1. np.array()

```
[2]: # Convert a list to array
my_list = [1,2,3666666,4,5]

# We can pass a list and specify a datatype (optional) in which we want to
↳store the value in array
arr_1d = np.array(my_list, np.int32)

arr_1d
```

```
[2]: array([    1,     2, 3666666,     4,     5], dtype=int32)
```

```
[3]: # Data type of the array
arr_1d.dtype
```

```
[3]: dtype('int32')
```

```
[4]: # Dimension of the array
arr_1d.shape # 1-Dimension array having 5 elements
```

```
[4]: (5,)
```

```
[5]: # new array
arr_2d = np.array([[1,2,3],[4,5,6]])
```

```
[6]: # Dimension of the array
arr_2d.shape # Rows = 2 & Cols = 3 --> 2 Dimension
```

```
[6]: (2, 3)
```

```
[7]: # Change shape
arr_2d.reshape(3,2) # we have to maintain the product of original array i.e.
↳2x3 = 6
```

```
[7]: array([[1, 2],
           [3, 4],
           [5, 6]])
```

```
[8]: # Change shape
arr_2d.reshape(1,6) # Product is 1x6 = 6
```

```
[8]: array([[1, 2, 3, 4, 5, 6]])
```

To Visualize the Difference between 1D & 2D

- 1D will have One Opening square brackets & One closing square brackets
- 2D will have Two Opening square brackets & Two closing square brackets

```
[9]: # Convert a tuple to array  
arr_from_tup = np.array((3.4,4.5,5.3,6.6))
```

```
[10]: # Shape of array  
arr_from_tup.shape
```

```
[10]: (4,)
```

```
[11]: # data Type of array  
arr_from_tup.dtype
```

```
[11]: dtype('float64')
```

0.3.2 2. np.arange()

We can create an array with a range of elements

```
[12]: # Create an array from 1 to 10  
np.arange(start=1, stop=10)
```

```
[12]: array([1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
[13]: # Create an array from 1 to 10 with step-size=2  
np.arange(start=1, stop=10, step=2)
```

```
[13]: array([1, 3, 5, 7, 9])
```

```
[14]: # Right now its 1D  
np.arange(1,10,2).shape
```

```
[14]: (5,)
```

```
[15]: # Convert to 2D  
np.arange(1,10,2).reshape(5,1).shape
```

```
[15]: (5, 1)
```

```
[16]: np.arange(50,1,-1)
```

```
[16]: array([50, 49, 48, 47, 46, 45, 44, 43, 42, 41, 40, 39, 38, 37, 36, 35, 34,  
        33, 32, 31, 30, 29, 28, 27, 26, 25, 24, 23, 22, 21, 20, 19, 18, 17,
```

```
16, 15, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2])
```

```
[17]: np.arange(1,18,3)
      np.arange(1,18,3).shape
      np.arange(1,18,3).reshape(2,3)
```

```
[17]: array([[ 1,  4,  7],
            [10, 13, 16]])
```

0.3.3 3. np.linspace()

We can create an array with values that are spaced linearly in a specified interval

```
[18]: # linspace -- equally spaced elements

      np.linspace(1,20,30) # 30 number of elements between 1 to 20 (including 1 and
      ↪ 20 both)
```

```
[18]: array([ 1.          ,  1.65517241,  2.31034483,  2.96551724,  3.62068966,
            4.27586207,  4.93103448,  5.5862069 ,  6.24137931,  6.89655172,
            7.55172414,  8.20689655,  8.86206897,  9.51724138, 10.17241379,
            10.82758621, 11.48275862, 12.13793103, 12.79310345, 13.44827586,
            14.10344828, 14.75862069, 15.4137931 , 16.06896552, 16.72413793,
            17.37931034, 18.03448276, 18.68965517, 19.34482759, 20.          ])
```

```
[19]: # reshaping
      np.linspace(1,20,30).reshape(3,10)
```

```
[19]: array([[ 1.          ,  1.65517241,  2.31034483,  2.96551724,  3.62068966,
            4.27586207,  4.93103448,  5.5862069 ,  6.24137931,  6.89655172],
            [ 7.55172414,  8.20689655,  8.86206897,  9.51724138, 10.17241379,
            10.82758621, 11.48275862, 12.13793103, 12.79310345, 13.44827586],
            [14.10344828, 14.75862069, 15.4137931 , 16.06896552, 16.72413793,
            17.37931034, 18.03448276, 18.68965517, 19.34482759, 20.          ]])
```

0.3.4 4. np.zeros()

We can create an array filled with 0's

```
[20]: # zeros
      np.zeros(3)
```

```
[20]: array([0., 0., 0.])
```

```
[21]: # zeros
      np.zeros((3,4))
```

```
[21]: array([[0., 0., 0., 0.],
           [0., 0., 0., 0.],
           [0., 0., 0., 0.]])
```

0.3.5 5. np.ones()

We can create an array filled with 1's

```
[22]: # ones
      np.ones(3)
```

```
[22]: array([1., 1., 1.])
```

```
[23]: # ones
      np.ones((3,4))
```

```
[23]: array([[1., 1., 1., 1.],
           [1., 1., 1., 1.],
           [1., 1., 1., 1.]])
```

```
[24]: # ones
      np.ones((5,2,8))
      # 2,8 denotes the inner most array of 2 rows and 8 columns
      # and there are 5 sets of 2,8 arrays making it 3D array
```

```
[24]: array([[[1., 1., 1., 1., 1., 1., 1., 1.],
             [1., 1., 1., 1., 1., 1., 1., 1.]],

            [[1., 1., 1., 1., 1., 1., 1., 1.],
             [1., 1., 1., 1., 1., 1., 1., 1.]],

            [[1., 1., 1., 1., 1., 1., 1., 1.],
             [1., 1., 1., 1., 1., 1., 1., 1.]],

            [[1., 1., 1., 1., 1., 1., 1., 1.],
             [1., 1., 1., 1., 1., 1., 1., 1.]],

            [[1., 1., 1., 1., 1., 1., 1., 1.],
             [1., 1., 1., 1., 1., 1., 1., 1.]])
```

0.3.6 6. np.empty()

We can create an array whose initial content is random and depends on the state of the memory.

The reason to use `empty` over `zeros` is speed - just make sure to fill every element afterwards!

```
[25]: # empty
      np.empty((3,7))
```

```
[25]: array([[0., 0., 0., 0., 0., 0., 0.],
            [0., 0., 0., 0., 0., 0., 0.],
            [0., 0., 0., 0., 0., 0., 0.]])
```

0.3.7 7. np.identity()

Return the identity array.

The identity array is a square array with ones on the main diagonal and others are zeros

```
[26]: # identity matrix of 4x4 shape
      np.identity(4)
```

```
[26]: array([[1., 0., 0., 0.],
            [0., 1., 0., 0.],
            [0., 0., 1., 0.],
            [0., 0., 0., 1.]])
```

0.4 Adding, Removing & Sorting elements

1. np.sort()
2. np.concatenate()

```
[27]: arr = np.array([4,3,6,7,1,9])
      print(arr)

      np.sort(arr)
```

```
[4 3 6 7 1 9]
```

```
[27]: array([1, 3, 4, 6, 7, 9])
```

```
[28]: # Concatenating same size of two array
      a = np.array([1,2,3,4])
      b = np.array([5,6,7,8])
      np.concatenate((a,b))
```

```
[28]: array([1, 2, 3, 4, 5, 6, 7, 8])
```

```
[29]: # Concatenating different sizes of array by axis=0
      a = np.array([[1,2], [3,4]])
      b = np.array([[5,6]])
      np.concatenate((a,b), axis=0)
```

```
[29]: array([[1, 2],
            [3, 4],
            [5, 6]])
```

```
[30]: # Concatenating different sizes of array by axis=1
a = np.array([[1,2], [3,4]])
b = np.array([[5,6]])
np.concatenate((a,b.T), axis=1)
```

```
[30]: array([[1, 2, 5],
           [3, 4, 6]])
```

0.5 Shape & Size of an Array

1. ndarray.ndim
2. ndarray.size
3. ndarray.shape

0.5.1 ndarray.ndim

tells us the number of axes, or dimensions, of the array.

```
[31]: arr = np.array([[0, 1, 2, 3], [4, 5, 6, 7]],
                    [[0, 1, 2, 3], [4, 5, 6, 7]],
                    [[0, 1, 2, 3], [4, 5, 6, 7]])
arr.ndim # gives the dimension of the array
```

```
[31]: 3
```

0.5.2 ndarray.size

finds total number of elements in the array

```
[32]: arr.size
```

```
[32]: 24
```

0.5.3 ndarray.shape

finds the shape of the array

```
[33]: arr.shape
```

```
[33]: (3, 2, 4)
```

0.6 Re-Shaping the Array

Note:- we can reshape an array to any dimension by keeping the size of its original array same

```
[34]: # arr.reshape()

arr = np.arange(12) # here size is 12
```

```

print(arr)
print()

# reshaping into 2D array of Rows=2 and Columns=6, Size = 12
print(arr.reshape(2,6))
print()

# reshaping into 2D array of Rows=4 and Columns=3, Size = 12
print(arr.reshape(4,3))
print()

# reshaping into 3D array of 3,2,2 maintaining the Size = 12
print(arr.reshape(3,2,2))
print()

```

```
[ 0  1  2  3  4  5  6  7  8  9 10 11]
```

```
[[ 0  1  2  3  4  5]
 [ 6  7  8  9 10 11]]
```

```
[[ 0  1  2]
 [ 3  4  5]
 [ 6  7  8]
 [ 9 10 11]]
```

```
[[[ 0  1]
   [ 2  3]]
```

```
[[ 4  5]
 [ 6  7]]
```

```
[[ 8  9]
 [10 11]]
```

0.7 NumPy Random Module

The `numpy.random` module implements pseudo-random number generators with the ability to draw samples from a variety of probability distributions.

0.7.1 `np.random.rand()`

Random number generation (Uniform distribution)

```
[35]: np.random.rand(2,3)
```

```
[35]: array([[0.0117202 , 0.86750348, 0.45139067],
            [0.1103309 , 0.30405034, 0.08778116]])
```


0.7.2 np.random.randn()

Random number generation (Normal distribution with Mean=0 & Standard Deviation=1)

```
[36]: np.random.randn(2,3)
```

```
[36]: array([[ 1.59436956,  1.65239378, -0.70732319],
          [-1.37190062, -1.47977959,  0.90136213]])
```

0.7.3 np.random.randint()

Return random integers from the “discrete uniform” distribution of the given size from low (inclusive) to high (exclusive)

```
[37]: np.random.randint(low=1, high=100, size=10)  # here size=10, which gives 10
        ↪ number between min and max limit
```

```
[37]: array([12, 45, 68, 54, 65, 43, 79, 94, 96, 28])
```

```
[38]: # we can create a n-dimensional array by specifying the size
        np.random.randint(1,100,(4,4))  # here it is 4x4 size of array having random
        ↪ values
```

```
[38]: array([[53,  6, 65, 69],
          [20, 82, 27, 55],
          [89, 98, 71, 34],
          [58, 23, 17, 36]])
```

0.7.4 np.random.random_sample()

Return random floats in the half-open interval [0.0, 1.0).

```
[72]: np.random.random_sample(size=(2,6))
```

```
[72]: array([[0.19465933, 0.72918811, 0.33219758, 0.04920512, 0.99888788,
          0.32102308],
          [0.08149279, 0.05946466, 0.27319728, 0.28875117, 0.55290726,
          0.19653602]])
```

0.7.5 np.random.normal()

Draw random samples from a normal (Gaussian) distribution

```
[89]: np.random.normal(loc=0, scale=1, size=15)  # 'loc' denotes mean & 'scale'
        ↪ denotes Standard Deviation for 15 numbers
```

```
[89]: array([ 1.40924246,  0.00650419,  0.5397556 , -2.49951693, -0.55381917,
          0.4406829 ,  0.27065829,  0.94675684,  1.95851061,  0.1540836 ,
          -0.88915799, -0.31680415, -0.12880158, -1.19021509,  0.25109025])
```

0.8 Indexing and slicing of Arrays

We can index and slice NumPy arrays in the same way we slice Python lists.

```
[39]: arr = np.arange(0,11)
      arr
```

```
[39]: array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10])
```

```
[40]: # data at position 1
      arr[1]
```

```
[40]: 1
```

```
[41]: # data from position 2 till the end of array
      arr[2:]
```

```
[41]: array([ 2,  3,  4,  5,  6,  7,  8,  9, 10])
```

```
[42]: # data between position 2 and 7 (exclusive)
      arr[2:7]
```

```
[42]: array([2, 3, 4, 5, 6])
```

```
[43]: # Negative indexing from last till 3rd last
      arr[-1:7:-1]
```

```
[43]: array([10,  9,  8])
```

```
[44]: # Negative indexing from last till 3rd last
      arr[-1:-4:-1]
```

```
[44]: array([10,  9,  8])
```

```
[45]: # print all of the values in the array that are less than equal to 5
      arr[arr<=5]
```

```
[45]: array([0, 1, 2, 3, 4, 5])
```

```
[46]: # select elements that satisfy two conditions using the & and / operators
      arr[(arr<=8) & (arr>2)]
```

```
[46]: array([3, 4, 5, 6, 7, 8])
```

0.8.1 Problem statement

```
[47]: arr1 = np.random.randint(2,100,20).reshape(5,4)
      arr1
```

```
[47]: array([[34, 40, 30, 23],
            [46, 85, 23, 56],
            [ 4, 23, 55,  6],
            [64, 94, 28, 84],
            [20, 48,  7, 87]])
```

```
[48]: # all data till 3rd row and all data till 3rd column
      arr1[:3,:3]
```

```
[48]: array([[34, 40, 30],
            [46, 85, 23],
            [ 4, 23, 55]])
```

```
[49]: # all data starting from 3rd row and all data starting from 1st column
      arr1[3:,1:]
```

```
[49]: array([[94, 28, 84],
            [48,  7, 87]])
```

```
[50]: # all data starting from 2nd row till 3rd row and all data of 1st and last
      ↪column
      arr1[2:4,[0,-1]]
```

```
[50]: array([[ 4,  6],
            [64, 84]])
```

0.9 Basic array operations

```
[51]: 5 + np.ones((3,5)) # add 5 to all elements in an array
```

```
[51]: array([[6., 6., 6., 6., 6.],
            [6., 6., 6., 6., 6.],
            [6., 6., 6., 6., 6.]])
```

```
[52]: arr1 = np.random.randint(1,20,(2,2))
      arr2 = np.random.randint(10,20,(2,2))
```

```
[53]: print(f"Array1: \n{arr1}")

      print(f"\nArray2: \n{arr2}")
```

```
Array1:
[[18 14]
```

```
[ 7  4]]
```

Array2:

```
[[15 10]
 [10 16]]
```

```
[54]: print(f"\n Addition: \n{arr1 + arr2}")
      print(f"\n Subtraction: \n{arr1 - arr2}")
      print(f"\n Multiplication: \n{arr1 * arr2}")
      print(f"\n Division: \n{arr1 / arr2}")
```

Addition:

```
[[33 24]
 [17 20]]
```

Subtraction:

```
[[ 3  4]
 [-3 -12]]
```

Multiplication:

```
[[270 140]
 [ 70  64]]
```

Division:

```
[[1.2  1.4 ]
 [0.7  0.25]]
```

```
[55]: # sum
      print(arr1)
      print("Sum without axis", arr1.sum())
      print("Sum axis=0", arr1.sum(axis=0))
      print("Sum axis=1", arr1.sum(axis=1))
```

```
[[18 14]
 [ 7  4]]
Sum without axis 43
Sum axis=0 [25 18]
Sum axis=1 [32 11]
```

```
[56]: # Addition
      arr1 + arr2
```

```
[56]: array([[33, 24],
            [17, 20]])
```

```
[57]: np.sqrt(arr1)
```

```
[57]: array([[4.24264069, 3.74165739],
           [2.64575131, 2.          ]])
```

```
[58]: np.exp(arr2)
```

```
[58]: array([[3269017.37247211, 22026.46579481],
           [ 22026.46579481, 8886110.52050787]])
```

```
[59]: np.log10(arr1)
```

```
[59]: array([[1.25527251, 1.14612804],
           [0.84509804, 0.60205999]])
```

0.10 How to create an array from existing data

0.10.1 1. Broadcasting

- Broadcasting is a mechanism that allows NumPy to perform operations on arrays of different shapes.
- The dimensions of the array must be compatible, for example, when the dimensions of both arrays are equal or when one of them is 1.
- If the dimensions are not compatible, you will get a `ValueError`.

```
[60]: # Broadcasting
arr = np.arange(1,10)
arr[2:] = 1000
print(arr)
```

```
[ 1  2 1000 1000 1000 1000 1000 1000 1000]
```

```
[61]: # Broadcasting / NumPy understands that the multiplication should happen with
      ↪ each cell. That concept is called broadcasting.
data = np.array([1.0, 2.0])
data * 1.6
```

```
[61]: array([1.6, 3.2])
```

0.10.2 2. Copying of Arrays

```
[62]: # Arrays are Reference typed, that means assigning one array to another also
      ↪ shares the same memory reference.add()
      # To prevent, the above, we must use copy() method

arr1 = arr.copy()
print(arr)
```

```
arr1[3:] = 500
print(arr1)
```

```
[ 1  2 1000 1000 1000 1000 1000 1000 1000]
[ 1  2 1000  500  500  500  500  500  500]
```

0.10.3 3. Stacking (Vertical & Horizontal)

```
[63]: arr1 = np.array([[1,1], [2,2]])
      arr2 = np.array([[3,3], [4,4]])

      print(f"Vertical Stack: \n{np.vstack((arr1, arr2))}")

      print(f"\nHorizontal Stack: \n{np.hstack((arr1, arr2))}")
```

Vertical Stack:

```
[[1 1]
 [2 2]
 [3 3]
 [4 4]]
```

Horizontal Stack:

```
[[1 1 3 3]
 [2 2 4 4]]
```

0.11 Get unique items and counts

```
[64]: arr = np.array([11, 11, 12, 13, 14, 15, 16, 17, 12, 13, 11, 14, 18, 19, 20])
      arr
```

```
[64]: array([11, 11, 12, 13, 14, 15, 16, 17, 12, 13, 11, 14, 18, 19, 20])
```

```
[73]: unique_values = np.unique(arr)  # alternatively, we can remove duplicates from
      ↪ the array
      unique_values
```

```
[73]: array([11, 12, 13, 14, 15, 16, 17, 18, 19, 20])
```

```
[75]: unique_values, index_positions = np.unique(arr, return_index=True)
      print(unique_values)
      print(index_positions)
```

```
[11 12 13 14 15 16 17 18 19 20]
[ 0  2  3  4  5  6  7 12 13 14]
```

```
[76]: unique_values, counts = np.unique(arr, return_counts=True)
      print(unique_values)
```

```
print(counts)
```

```
[11 12 13 14 15 16 17 18 19 20]
[3 2 2 2 1 1 1 1 1 1]
```

1 NumPy: Exercises

```
[79]: # Write a NumPy program to create an array of all even integers from 30 to 70.

np.arange(start=30, stop=70, step=2)
```

```
[79]: array([30, 32, 34, 36, 38, 40, 42, 44, 46, 48, 50, 52, 54, 56, 58, 60, 62,
        64, 66, 68])
```

```
[80]: # Write a NumPy program to create a 3x3 identity matrix.

np.identity(3)
```

```
[80]: array([[1., 0., 0.],
        [0., 1., 0.],
        [0., 0., 1.]])
```

```
[87]: # Write a NumPy program to generate an array of 15 random numbers from a
      ↪ standard normal distribution.
np.random.normal(loc=0, scale=1, size=15) # 'loc' denotes mean & 'scale'
      ↪ denotes Standard Deviation
```

```
[87]: array([ 0.15030259,  1.30645184, -0.28881421, -0.99897669, -0.41408066,
        -1.83996599, -1.37655922,  0.74360087,  0.11151737, -0.23403126,
         0.12432222,  0.56679231,  0.13377066,  0.70120379, -0.90153684])
```

```
[98]: # Write a NumPy program to create a 2D array with 1 on the border and 0 inside.
arr = np.ones((5,5))
arr[1:-1, 1:-1] = 0
arr
```

```
[98]: array([[1., 1., 1., 1., 1.],
        [1., 0., 0., 0., 1.],
        [1., 0., 0., 0., 1.],
        [1., 0., 0., 0., 1.],
        [1., 1., 1., 1., 1.]])
```

```
[106]: # Write a NumPy program to test whether each element of a 1-D array is also
      ↪ present in a second array.
arr1 = np.array([ 0, 10, 20, 40, 60])
arr2 = np.array([0, 40])
# for i in range(len(arr1)):
```

```

#     if arr1[i] in arr2:
#         arr1[i] = True
#     else:
#         arr1[i] = False
print("Compare each element of array1 and array2")
np.in1d(arr1, arr2)

```

Compare each element of array1 and array2

[106]: array([True, False, False, True, False])

```

[118]: # Write a Python program to find the maximum and minimum value of a given
        ↪ flattened array.
arr = np.array([[0, 1],
                [2, 3]])
print(f"Max: {np.amax(arr)}")
print(f"Min: {np.amin(arr)}")

```

Max: 3

Min: 0